Shiny + Conexión de R con BBDD

Material teórico-práctico (8h)

Objetivo: construir apps Shiny desde cero y conectar R con bases de datos de forma segura usando DBI (y pool para producción). Incluye ejemplos completos y ejercicios.

0. Preparación

```
install.packages(c("shiny", "DBI", "RSQLite", "pool", "dplyr", "DT"))
library(shiny)
library(DBI)
library(RSQLite)
library(pool)
library(dplyr)
library(DT)
```

1. Shiny: fundamentos

1.1. Estructura mínima: ui + server + shinyApp()

```
library(shiny)

ui <- fluidPage(
  h2("Hola Shiny"),
  textInput("nombre", "Tu nombre:", value = "Ana"),
  textOutput("saludo")
)

server <- function(input, output, session) {
  output$saludo <- renderText({
    paste0("Hola ", input$nombre, "!")
  })
}

shinyApp(ui, server)
```

Tip: Shiny es reactivo: cuando cambia input$nombre, se recalcula renderText().

1.2. Inputs y outputs comunes

- Inputs: textInput, numericInput, sliderInput, selectInput, dateInput, fileInput, actionButton.
- Outputs: textOutput, tableOutput, plotOutput, DTOutput (tablas interactivas).

1.3. Reactividad: reactive(), observeEvent(), eventReactive()

Reglas rápidas:

- reactive(): define un valor calculado que se memoiza y se invalida cuando cambian dependencias.
- observe()/observeEvent(): ejecuta efectos colaterales (guardar, notificar, actualizar UI).
- eventReactive(): recalcula solo cuando ocurre un evento (por ejemplo, click en botón).

```
ui <- fluidPage(
```

```
  sliderInput("n", "N:", min = 10, max = 200, value = 50),
  actionButton("go", "Generar"),
  plotOutput("hist")
)

server <- function(input, output, session) {

  datos <- eventReactive(input$go, {
    rnorm(input$n)
  })

  output$hist <- renderPlot({
    hist(datos(), main = "Histograma", xlab = "valor")
  })
}

shinyApp(ui, server)
```

Tip: Usa eventReactive cuando NO quieres recalcular con cada cambio del slider, solo al presionar el botón.

1.4. Estado: reactiveVal y reactiveValues

```
server <- function(input, output, session) {
  contador <- reactiveVal(0)

  observeEvent(input$go, {
    contador(contador() + 1)
  })

  output$salida <- renderText({
    paste("Clicks:", contador())
  })
}
```

1.5. Validación: req() y validate()

```
output$res <- renderText({
  req(input$nombre)  # si está vacío, no sigue
  validate(
    need(nchar(input$nombre) >= 3, "El nombre debe tener al menos 3
caracteres.")
  )
  paste("OK:", input$nombre)
})
```

1.6. Layouts básicos

```
ui <- fluidPage(
  sidebarLayout(
    sidebarPanel(
      selectInput("var", "Variable:", choices = names(mtcars)),
      sliderInput("bins", "Bins:", min = 5, max = 50, value = 20)
    ),
    mainPanel(
      plotOutput("p"),
      DTOutput("tabla")
    )
  )
)
```

2. App ejemplo: explorar mtcars

```
library(shiny)
library(DT)

ui <- fluidPage(
  titlePanel("Explorador mtcars"),
  sidebarLayout(
    sidebarPanel(
      selectInput("x", "Eje X:", choices = names(mtcars), selected = "wt"),
      selectInput("y", "Eje Y:", choices = names(mtcars), selected = "mpg")
    ),
    mainPanel(
      plotOutput("scatter"),
      DTOutput("tabla")
    )
  )
)

server <- function(input, output, session) {

  output$scatter <- renderPlot({
    plot(mtcars[[input$x]], mtcars[[input$y]],
         xlab = input$x, ylab = input$y, pch = 19)
  })

  output$tabla <- renderDT({
    datatable(mtcars, options = list(pageLength = 10))
  })
}

shinyApp(ui, server)
```

3. Conexión de R con BBDD usando PostgreSQL (DBI + RPostgres)

3.1. Conceptos: conexión, consultas, tablas

- DBI define una interfaz común para conectar R con bases de datos. En este bloque usamos PostgreSQL con el driver RPostgres.
- Una conexión (con) se abre con dbConnect() y se cierra con dbDisconnect().
- Consultas: dbGetQuery() (traer resultados a R), dbExecute() (instrucciones sin resultado), dbWriteTable() (cargar tablas), dbReadTable() (leer tablas).
- Buenas prácticas: consultas parametrizadas (evitar SQL injection) y credenciales fuera del código (variables de entorno).

Paquetes usados:

```
install.packages(c('DBI','RPostgres','pool','glue','DT','shiny'))

library(DBI)
library(RPostgres)
```

3.2. Demo con PostgreSQL (local o servidor)

Este ejemplo crea una tabla temporal (TEMP) para no ensuciar el esquema. Necesitás tener un servidor PostgreSQL accesible (local o remoto).

Recomendado: configurar variables de entorno (en .Renviron o en el sistema):

```
Sys.setenv(
  PGHOST = 'localhost',
  PGPORT = '5432',
  PGDATABASE = 'mi_base',
  PGUSER = 'mi_usuario',
  PGPASSWORD = 'mi_password',
  PGSSLMODE = 'prefer'  # o 'require' si tu servidor lo exige
)
```

Conectar, crear tabla temporal, insertar y consultar:

```
library(DBI)
library(RPostgres)

con <- dbConnect(
  RPostgres::Postgres(),
  host     = Sys.getenv("PGHOST"),
  port     = as.integer(Sys.getenv("PGPORT", "5432")),
  dbname   = Sys.getenv("PGDATABASE"),
  user     = Sys.getenv("PGUSER"),
  password = Sys.getenv("PGPASSWORD"),
  sslmode  = Sys.getenv("PGSSLMODE", "prefer")
)

# 1) Tabla TEMP (vive solo durante la sesión)
dbExecute(con, "
  CREATE TEMP TABLE ventas (
```

```
    id      INTEGER,
    cliente TEXT,
    importe NUMERIC
  )
")

# 2) Cargar datos de ejemplo
dbExecute(con, "
  INSERT INTO ventas (id, cliente, importe) VALUES
  (1, 'Ana', 120.5),
  (2, 'Luis', 90.0),
  (3, 'Ana', 50.0),
  (4, 'Sofía', 200.0)
")

# 3) Consulta agregada
dbGetQuery(con, "
  SELECT cliente, SUM(importe) AS total
  FROM ventas
  GROUP BY cliente
  ORDER BY total DESC
")

dbDisconnect(con)
```

Tip: si tu query tarda o devuelve muchos datos, preferí dbSendQuery() + dbFetch() para traer resultados por partes.

3.3. Consultas parametrizadas (seguridad)

Nunca concatentes strings con inputs de usuario para construir SQL (riesgo de SQL injection). En PostgreSQL, una forma común es usar placeholders $1, $2, ... y pasar params.

Opción A (parametrización nativa con placeholders $1):

```
cliente <- "Ana"

dbGetQuery(
  con,
  "SELECT * FROM ventas WHERE cliente = $1",
  params = list(cliente)
)
```

Opción B (muy práctica y portable): sqlInterpolate() arma el SQL con quoting seguro usando la conexión.

```
cliente <- "Ana"

sql <- sqlInterpolate(
  con,
  "SELECT * FROM ventas WHERE cliente = ?cliente",
  cliente = cliente
)
```

```
dbGetQuery(con, sql)
```

Tip: si PostgreSQL no infiere el tipo de un parámetro (poco común), podés castear: $1::int, $1::text, etc.

3.4. Credenciales seguras

- No hardcodees usuario/password en el código.
- Usá variables de entorno (Sys.getenv) y un archivo .Renviron fuera del control de versiones.
- En servidores: secretos gestionados (Vault, AWS Secrets Manager, etc.).

Ejemplo de conexión usando env vars (Postgres):

```
con <- dbConnect(
  RPostgres::Postgres(),
  host     = Sys.getenv("PGHOST"),
  port     = as.integer(Sys.getenv("PGPORT", "5432")),
  dbname   = Sys.getenv("PGDATABASE"),
  user     = Sys.getenv("PGUSER"),
  password = Sys.getenv("PGPASSWORD"),
  sslmode  = Sys.getenv("PGSSLMODE", "prefer")
)
```

3.5. pool: conexiones reutilizables (recomendado en Shiny)

En apps Shiny, abrir/cerrar conexiones en cada consulta puede ser lento. pool mantiene conexiones disponibles y maneja la concurrencia.

```
library(pool)
library(DBI)
library(RPostgres)

pool_con <- dbPool(
  drv      = RPostgres::Postgres(),
  host     = Sys.getenv("PGHOST"),
  port     = as.integer(Sys.getenv("PGPORT", "5432")),
  dbname   = Sys.getenv("PGDATABASE"),
  user     = Sys.getenv("PGUSER"),
  password = Sys.getenv("PGPASSWORD"),
  sslmode  = Sys.getenv("PGSSLMODE", "prefer")
)

# Usar el pool (se comporta como una conexión DBI)
dbGetQuery(pool_con, "SELECT 1 AS ok")

# Cerrar al finalizar (en onStop o al cerrar la app)
poolClose(pool_con)
```

4. Integrar Shiny + PostgreSQL: app ejemplo (visor de tabla)

Esta app consulta una tabla ventas en PostgreSQL y permite filtrar por cliente. Usa pool para reutilizar conexiones.

Nota: para el ejemplo se incluye un bloque opcional de inicialización (crear tabla si no existe y cargar datos). En producción, la tabla suele existir y se omite esa parte.

```
library(shiny)
library(DBI)
library(RPostgres)
library(pool)
library(DT)

# --- Pool global (recomendado) ---
pool_con <- dbPool(
  drv      = RPostgres::Postgres(),
  host     = Sys.getenv("PGHOST"),
  port     = as.integer(Sys.getenv("PGPORT", "5432")),
  dbname   = Sys.getenv("PGDATABASE"),
  user     = Sys.getenv("PGUSER"),
  password = Sys.getenv("PGPASSWORD"),
  sslmode  = Sys.getenv("PGSSLMODE", "prefer")
)

# Cerrar pool cuando termina la app
onStop(function() poolClose(pool_con))

# --- (Opcional) Inicialización demo ---
dbExecute(pool_con, "
  CREATE TABLE IF NOT EXISTS ventas (
    id      INTEGER PRIMARY KEY,
    cliente TEXT NOT NULL,
    importe NUMERIC NOT NULL
  )
")

# Cargar datos solo si la tabla está vacía
n <- dbGetQuery(pool_con, "SELECT COUNT(*) AS n FROM ventas")$n[1]
if (n == 0) {
  dbExecute(pool_con, "
    INSERT INTO ventas (id, cliente, importe) VALUES
    (1, 'Ana', 120.5),
    (2, 'Luis', 90.0),
    (3, 'Ana', 50.0),
    (4, 'Sofía', 200.0)
  ")
}

ui <- fluidPage(
  titlePanel("Visor de ventas (PostgreSQL)"),
  sidebarLayout(
    sidebarPanel(
      textInput('cliente', 'Filtrar cliente (opcional):', value = ''),
      actionButton('buscar', 'Buscar')
    ),
```

```
    mainPanel(
      DTOutput('tabla')
    )
  )
)

server <- function(input, output, session) {

  consulta <- eventReactive(input$buscar, {
    cli <- trimws(input$cliente)

    if (nchar(cli) == 0) {
      dbGetQuery(pool_con, "SELECT * FROM ventas ORDER BY id")
    } else {
      # Parametrizada con $1 (Postgres)
      dbGetQuery(
        pool_con,
        "SELECT * FROM ventas WHERE cliente = $1 ORDER BY id",
        params = list(cli)
      )
    }
  }, ignoreInit = TRUE)

  output$tabla <- renderDT({
    req(consulta())
    datatable(consulta(), options = list(pageLength = 10))
  })
}

shinyApp(ui, server)
```

Ejercicios (para practicar)

1) Consultas SQL básicas

1. Traé solo las ventas con importe > 100.
2. Mostrá total vendido por cliente, ordenado de mayor a menor.
3. Agregá un filtro por importe mínimo en la app Shiny (input numérico) y armá la query parametrizada.

2) Escritura/actualización

4. Insertá una nueva venta desde R (dbExecute) con parámetros (id, cliente, importe).
5. Actualizá el importe de una venta por id.
6. Borrá una venta por id (con confirmación en Shiny).

3) Buenas prácticas

7. Mové el bloque de inicialización demo a un script separado (por ejemplo init_db.R).
8. Guardá credenciales en .Renviron y verificá que el repo no contiene passwords.
9. Agregá un índice en la columna cliente y medí diferencia si la tabla crece (EXPLAIN ANALYZE).

5. Ejercicios

Ejercicio 1

Crea una app Shiny con un sliderInput para elegir N y muestra un histograma de rnorm(N).

Tip: sliderInput + renderPlot.

Ejercicio 2

Añade un actionButton para recalcular los datos solo al pulsar (eventReactive).

Tip: eventReactive(input$boton, {...}).

Ejercicio 3

Crea una app que filtre mtcars por un rango de mpg y muestre la tabla filtrada.

Tip: sliderInput(range) + filter.

Ejercicio 4

Crea una base SQLite local con una tabla clientes y otra ventas; escribe ambas con dbWriteTable.

Tip: dbWriteTable(con, 'clientes', df).

6. Soluciones propuestas (esqueleto)

Solución 1

Histograma con slider:

```
library(shiny)

ui <- fluidPage(
  sliderInput("n", "N:", 10, 500, 50),
  plotOutput("p")
)

server <- function(input, output, session) {
  output$p <- renderPlot({
    hist(rnorm(input$n), main = "rnorm", xlab = "")
  })
}

shinyApp(ui, server)
```

Solución 2

Recalcular con botón:

```
library(shiny)
```

```
ui <- fluidPage(
  sliderInput("n", "N:", 10, 500, 50),
  actionButton("go", "Generar"),
  plotOutput("p")
)

server <- function(input, output, session) {
  datos <- eventReactive(input$go, { rnorm(input$n) }, ignoreInit = TRUE)

  output$p <- renderPlot({
    req(datos())
    hist(datos(), main = "rnorm", xlab = "")
  })
}

shinyApp(ui, server)
```

Solución 3

Filtrar mtcars por mpg:

```
library(shiny)
library(DT)

ui <- fluidPage(
  sliderInput("mpg", "Rango mpg:", min(mtcars$mpg), max(mtcars$mpg),
              value = range(mtcars$mpg)),
  DTOutput("tabla")
)

server <- function(input, output, session) {
  output$tabla <- renderDT({
    datatable(mtcars[mtcars$mpg >= input$mpg[1] & mtcars$mpg <= input$mpg[2],
],
              options = list(pageLength = 10))
  })
}

shinyApp(ui, server)
```

Solución 4

Crear SQLite y escribir tablas:

```
library(DBI)
library(RSQLite)

con <- dbConnect(SQLite(), "mini.sqlite")

clientes <- data.frame(cliente = c("Ana","Luis","Sofía"), edad = c(30, 28,
35))
```

```
ventas <- data.frame(id = 1:4, cliente = c("Ana","Luis","Ana","Sofía"),
importe = c(120.5, 90, 50, 200))

dbWriteTable(con, "clientes", clientes, overwrite = TRUE)
dbWriteTable(con, "ventas", ventas, overwrite = TRUE)

dbListTables(con)
dbDisconnect(con)
```